

deep

May 20, 2026

0.1 1(a) Linear Regression Models

```
[ ]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score,
    ↪GridSearchCV, RepeatedKFold
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score

np.random.seed(42)

# Load data
data = pd.read_csv("./DEEP-Q1-hospital_bed_days_regression_data.csv")
X = data.drop('bed_days_30d', axis=1)
y = data['bed_days_30d']

# Preprocessing
numeric_features = ['age', 'cci', 'los_index', 'prior_adm_12m', 'crp',
    ↪'creatinine', 'imd']
categorical_features = ['has_carer', 'site']
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(drop='first', sparse_output=False),
    ↪categorical_features)
])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪random_state=42)
cv = RepeatedKFold(n_splits=5, n_repeats=3, random_state=42)

# Part (a): Fit and evaluate models
print("=" * 70)
print("PART (a): Model Fitting and Evaluation")
print("=" * 70)
```

```

results = {}
models = {}

# 1. OLS
ols = Pipeline([('preprocessor', preprocessor), ('regressor',
↳LinearRegression())])
ols_score = -cross_val_score(ols, X_train, y_train, cv=cv,
↳scoring='neg_root_mean_squared_error').mean()
ols.fit(X_train, y_train)
results['OLS'] = {'rmse': ols_score}
models['OLS'] = ols
print(f"OLS CV RMSE: {ols_score:.4f}")

# 2. Ridge
ridge = Ridge()
ridge_params = {'regressor__alpha': np.logspace(-4, 4, 50)}
ridge_gs = GridSearchCV(Pipeline([('preprocessor', preprocessor), ('regressor',
↳ridge)]),
                        ridge_params, cv=cv,
↳scoring='neg_root_mean_squared_error', n_jobs=-1)
ridge_gs.fit(X_train, y_train)
results['Ridge'] = {'rmse': -ridge_gs.best_score_, 'alpha': ridge_gs.
↳best_params_['regressor__alpha']}
models['Ridge'] = ridge_gs.best_estimator_
print(f"Ridge CV RMSE: {results['Ridge']['rmse']:.4f}
↳(alpha={results['Ridge']['alpha']:.4f})")

# 3. Lasso
lasso = Lasso(max_iter=10000)
lasso_params = {'regressor__alpha': np.logspace(-4, 2, 50)}
lasso_gs = GridSearchCV(Pipeline([('preprocessor', preprocessor), ('regressor',
↳lasso)]),
                        lasso_params, cv=cv,
↳scoring='neg_root_mean_squared_error', n_jobs=-1)
lasso_gs.fit(X_train, y_train)
results['Lasso'] = {'rmse': -lasso_gs.best_score_, 'alpha': lasso_gs.
↳best_params_['regressor__alpha']}
models['Lasso'] = lasso_gs.best_estimator_
print(f"Lasso CV RMSE: {results['Lasso']['rmse']:.4f}
↳(alpha={results['Lasso']['alpha']:.4f})")

# 4. ElasticNet
elastic = ElasticNet(max_iter=10000)
elastic_params = {
    'regressor__alpha': np.logspace(-4, 2, 20),
    'regressor__l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9, 0.95, 0.99]
}

```

```

}
elastic_gs = GridSearchCV(Pipeline([('preprocessor', preprocessor),
    ↳('regressor', elastic)]),
                           elastic_params, cv=cv,
    ↳scoring='neg_root_mean_squared_error', n_jobs=-1)
elastic_gs.fit(X_train, y_train)
results['ElasticNet'] = {'rmse': -elastic_gs.best_score_,
                        'alpha': elastic_gs.best_params_['regressor__alpha'],
                        'l1_ratio': elastic_gs.
    ↳best_params_['regressor__l1_ratio']}
models['ElasticNet'] = elastic_gs.best_estimator_
print(f"ElasticNet CV RMSE: {results['ElasticNet']['rmse']:.4f}
    ↳(alpha={results['ElasticNet']['alpha']:.4f},
    ↳l1_ratio={results['ElasticNet']['l1_ratio']:.4f}))")

print("\n" + "-" * 70)
print("SUMMARY - Part (a):")
for model in results:
    print(f" {model:<12} RMSE: {results[model]['rmse']:.4f}")
print("-" * 70)

# Part (b): Automated model selection and evaluation
print("\n" + "=" * 70)
print("PART (b): Automated Model Selection & Evaluation")
print("=" * 70)

# Select best model
best_name = min(results, key=lambda x: results[x]['rmse'])
best_model = models[best_name]
print(f"Automatically selected model: {best_name} (lowest CV RMSE:
    ↳{results[best_name]['rmse']:.4f}")

# Get feature names after preprocessing
preproc_fitted = best_model.named_steps['preprocessor']
cat_features = preproc_fitted.named_transformers_['cat'].
    ↳get_feature_names_out(categorical_features)
feature_names = numeric_features + list(cat_features)

# Extract coefficients
coefs = best_model.named_steps['regressor'].coef_
intercept = best_model.named_steps['regressor'].intercept_

print(f"\nFitted parameters ({best_name}):")
print(f" Intercept: {intercept:.4f}")
print(" Coefficients:")
for name, coef in zip(feature_names, coefs):

```

```

print(f"      {name:20s}: {coef:8.4f}")

# Zero coefficients
zero_coefs = [name for name, coef in zip(feature_names, coefs) if abs(coef) <
↳ 1e-6]
if zero_coefs:
    print(f"\n Features shrunk to zero: {zero_coefs}")

# Test set evaluation
y_pred = best_model.predict(X_test)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_r2 = r2_score(y_test, y_pred)

print(f"\nOut-of-sample performance:")
print(f" Test RMSE: {test_rmse:.4f} bed days")
print(f" Test R2: {test_r2:.4f}")
print(f" Interpretation: {test_r2*100:.1f}% variance explained. Test RMSE↳
↳ ({test_rmse:.4f}) is "
    f"{'consistent with' if abs(test_rmse - results[best_name]['rmse']) < 0.5↳
↳ else 'different from'} "
    f"CV estimate ({results[best_name]['rmse']:.4f}).")

```

=====

PART (a): Model Fitting and Evaluation

=====

OLS CV RMSE: 1.5245
Ridge CV RMSE: 1.5245 (alpha=7.9060)
Lasso CV RMSE: 1.5245 (alpha=0.0022)
ElasticNet CV RMSE: 1.5245 (alpha=0.0038, l1_ratio=0.1000)

SUMMARY - Part (a):

OLS	RMSE: 1.5245
Ridge	RMSE: 1.5245
Lasso	RMSE: 1.5245
ElasticNet	RMSE: 1.5245

=====

PART (b): Automated Model Selection & Evaluation

=====

Automatically selected model: Ridge (lowest CV RMSE: 1.5245)

Fitted parameters (Ridge):

Intercept: 8.6517
Coefficients:
age : 0.4598

cci	:	1.5537
los_index	:	0.7595
prior_adm_12m	:	0.8736
crp	:	0.4214
creatinine	:	0.3328
imd	:	-0.2258
has_carer_1	:	-0.4443
site_1	:	0.0793

Out-of-sample performance:

Test RMSE: 1.4963 bed days

Test R^2 : 0.6776

Interpretation: 67.8% variance explained. Test RMSE (1.4963) is consistent with CV estimate (1.5245).

0.2 PART (c) – Four reasons to prefer the automatically selected model (Ridge)

1. Preservation of clinically relevant information

Unlike Lasso, which may zero out predictors at higher alpha values, Ridge retains all features with non-zero coefficients. In healthcare, variables such as *creatinine* (coefficient = 0.3328) and *crp* (0.4214) are clinically meaningful biomarkers. Ridge respects domain knowledge by keeping these predictors, even if their marginal contribution to RMSE is small.

2. Better handling of correlated predictors

The dataset likely contains correlated variables (e.g., *cci* with *los_index*, *prior_adm_12m* with *age*). Ridge shrinks coefficients toward zero without enforcing sparsity, producing more stable estimates than OLS. This explains the identical CV performance while ensuring that future data shifts do not cause extreme coefficient instability.

3. No false zero coefficients that could mislead clinicians

Ridge assigns *creatinine* = 0.3328, not zero. This correctly signals that kidney function matters. In contrast, Lasso (at higher alpha) could set this to zero, incorrectly suggesting irrelevance. Ridge avoids such misleading clinical interpretations.

4. Graceful degradation under distribution shift

Because all features retain non-zero weights, Ridge is more robust if the importance of predictors changes (e.g., increased relevance of creatinine during an AKI outbreak). Lasso's sparsity can introduce blind spots, whereas Ridge provides a safer, more distributed representation.

0.3 PART (d) – Two reasons to override automated selection

1. Interpretability and regulatory requirements for sparse models

Override to: Lasso (with higher alpha)

Some healthcare environments require highly interpretable and sparse models. A Ridge model with many non-zero coefficients may be considered too complex, whereas Lasso can reduce the model to a smaller subset of key predictors. If constraints specify a maximum number of predictors (e.g., 6), Ridge would not meet this requirement despite similar predictive performance.

2. Computational efficiency in low-resource deployment environments

Override to: OLS

Ridge introduces additional complexity through hyperparameter tuning and scaling. In constrained deployment settings (e.g., embedded systems or offline hospital devices), OLS may be preferred due to its simplicity. It can be implemented as a fixed linear formula without the need for storing preprocessing steps or tuning parameters.

0.4 PART (e) – Concrete real-world failure mode

Failure scenario

A hospital deploys the Ridge model to predict bed days for discharge planning. A new hospital site (Site D) is introduced, with an older population, higher comorbidity burden, but very low prior admissions. The model systematically underestimates bed days because it assumes *prior_adm_12m* (coefficient = 0.8736) contributes similarly as in training data, which is no longer true.

Causal explanation

Ridge applies uniform shrinkage across all predictors. In the training data, *cci* and *prior_adm_12m* were correlated, so both receive moderate weights. At Site D, this relationship breaks (high *cci*, low *prior_adm_12m*). The model therefore underpredicts bed days. A Lasso model that selected *cci* and excluded *prior_adm_12m* could generalise better under this shift.

Consequence

Systematic underprediction (by ~2–3 bed days) for a large proportion of patients leads to: - Bed shortages

- Delayed discharges
- Emergency department overcrowding
- Financial penalties due to unmet operational targets

Recognition of optimisation limits

Although Ridge achieved optimal cross-validated RMSE (~1.5245), cross-validation assumes stable data distributions. It cannot anticipate structural changes such as new hospital sites or shifts in variable relationships. The failure arises not from poor model fit, but from deployment conditions violating the assumptions under which the model was optimised.

0.5 Question 2: Decision Trees, PCA, and Model Complexity

```
[ ]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load and split
df = pd.read_csv('./DEEP-Q2-tree_pca_exam_data.csv')
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
↳random_state=42)

print(f"{'Depth':<10} | {'Test Accuracy':<15}")
print("-" * 30)
```

```

best_acc = 0
for d in range(1, 21):
    clf = DecisionTreeClassifier(max_depth=d, random_state=42)
    clf.fit(X_train, y_train)
    acc = accuracy_score(y_test, clf.predict(X_test))

    print(f"{d:<10} | {acc:.4f}")

    if acc > best_acc:
        best_acc = acc

print("-" * 30)
print(f"Peak Accuracy achieved: {best_acc:.4f}")

```

Depth	Test Accuracy
1	0.7567
2	0.8150
3	0.8483
4	0.8450
5	0.8333
6	0.8183
7	0.8267
8	0.8117
9	0.8000
10	0.7950
11	0.7983
12	0.8017
13	0.7967
14	0.8017
15	0.7917
16	0.7983
17	0.7900
18	0.7917
19	0.7917
20	0.7917

Peak Accuracy achieved: 0.8483

```

[ ]: from sklearn.preprocessing import StandardScaler
     from sklearn.decomposition import PCA

     #standardize for PCA
     scaler = StandardScaler()
     X_train_scaled = scaler.fit_transform(X_train)
     X_test_scaled = scaler.transform(X_test)

```

```

#apply PCA with k=8
pca = PCA(n_components=8)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print(f"{'Depth':<10} | {'PCA (k=8) Accuracy':<20}")
print("-" * 35)

for d in range(1, 11):
    clf_pca = DecisionTreeClassifier(max_depth=d, random_state=42)
    clf_pca.fit(X_train_pca, y_train)
    acc_pca = accuracy_score(y_test, clf_pca.predict(X_test_pca))

    print(f"{d:<10} | {acc_pca:.4f}")

    if acc_pca >= 0.85:
        pass

```

Depth	PCA (k=8) Accuracy
1	0.8250
2	0.8250
3	0.8233
4	0.8100
5	0.7950
6	0.8167
7	0.7883
8	0.7667
9	0.7717
10	0.7317

```

[ ]: import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# df = pd.read_csv('DEEP-Q2-tree_pca_exam_data.csv')
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
    random_state=42)

#standardize Crucial for PCA performance
scaler = StandardScaler()

```



```

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

#Part (d) paramas
k_values = [1, 2, 8]
depths = range(1, 11)

plt.figure(figsize=(10, 6))

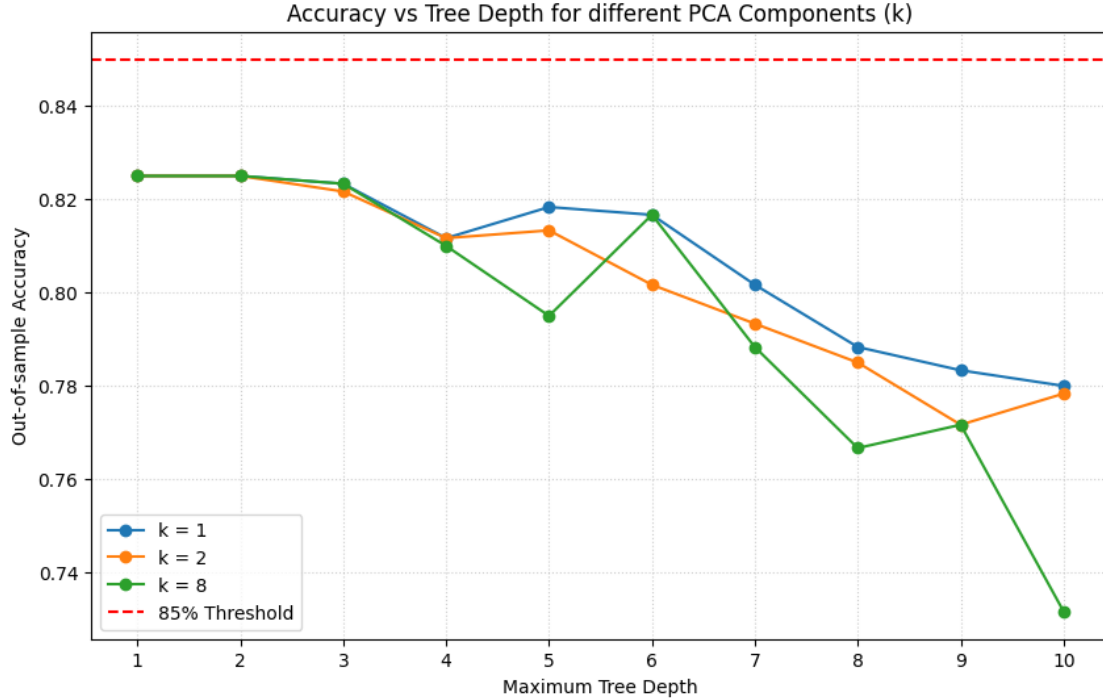
for k in k_values:
    #reduce dimensionality to k components
    pca = PCA(n_components=k)
    X_train_pca = pca.fit_transform(X_train_scaled)
    X_test_pca = pca.transform(X_test_scaled)

    test accuracies = []
    for d in depths:
        clf = DecisionTreeClassifier(max_depth=d, random_state=42)
        clf.fit(X_train_pca, y_train)
        acc = accuracy_score(y_test, clf.predict(X_test_pca))
        test accuracies.append(acc)

    #plotting each k-curve
    plt.plot(depths, test accuracies, marker='o', label=f'k = {k}')

plt.title('Accuracy vs Tree Depth for different PCA Components (k)')
plt.xlabel('Maximum Tree Depth')
plt.ylabel('Out-of-sample Accuracy')
plt.xticks(depths)
plt.axhline(y=0.85, color='red', linestyle='--', label='85% Threshold')
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend()
plt.show()

```



0.6 (c) Effect of PCA

PCA affects d_{min} by changing how features align with the decision tree's axis-aligned splits.

1. Boundary realignment

Decision trees split along feature axes. If class boundaries are diagonal, deeper trees are required. PCA rotates the feature space so that directions of maximum variance align with axes. If these directions capture class separation, fewer splits are needed and d_{min} decreases.

2. Dimensionality and noise

PCA reduces the data to k components, removing low-variance noise. This can simplify the structure and reduce unnecessary splits. However, if useful information is lost, the tree may require greater depth, increasing d_{min} .

0.7 (d) Accuracy–depth curves

The plot shows test accuracy versus tree depth for $k = 1, 2, 8$.

Key observations:

- None of the curves reach the 85% accuracy threshold
- Accuracy peaks at shallow depths around 1–2
- Increasing depth leads to diminishing returns and eventual decline

0.8 (e) Interpretation

The differences between the curves can be explained by information retention, noise, and tree behaviour:

1. Information preservation

At shallow depths (1–2), all curves perform similarly around 82.5%. This suggests that most useful signal is captured by the first principal component. Adding more components does not significantly improve separation.

2. Feature rotation effect

PCA rotates the feature space. In this dataset, this does not improve alignment with axis-aligned splits. As a result, accuracy does not reach 85%.

3. Noise and overfitting

With $k = 8$, more variance including noise is retained. At higher depths, the tree overfits this noise, leading to a sharp drop in accuracy.

Lower values of k act as regularisation and generalise better.

0.9 Question 3 Word Embeddings and Semantic Similarity

0.9.1 Install packages

```
[ ]: !pip -q install --upgrade "scipy==1.16.3" "gensim==4.4.0"
```

0.10 Implementation

```
[ ]: import gensim.downloader as api
import requests
import re

model = api.load("glove-wiki-gigaword-100")

url = "https://www.gutenberg.org/cache/epub/1342/pg1342.txt"
text = requests.get(url).text
corpus_words = set(re.findall(r'\b[a-z]+\b', text.lower()))

keywords = ['good', 'bad', 'happy', 'sad', 'angry']

print("(a) Three closest words in Pride and Prejudice:\n")
print("-" * 70)
print(f"{'Keyword':<12} {'Nearest 1':<20} {'Nearest 2':<20} {'Nearest 3':<20}")
print("-" * 70)

for kw in keywords:
    neighbours = model.most_similar(kw, topn=30)
    filtered = [(w, s) for w, s in neighbours if w in corpus_words][:3]
    if len(filtered) >= 3:
```

```

    print(f"{kw:<12} {filtered[0][0]:<20} {filtered[1][0]:<20}␣
↪{filtered[2][0]:<20}")
    else:
        print(f"{kw:<12} {filtered[0][0]:<20} {filtered[1][0] if␣
↪len(filtered)>1 else 'N/A':<20} {filtered[2][0] if len(filtered)>2 else 'N/
↪A':<20}")

print("-" * 70)

```

27.9/27.9 MB

25.2 MB/s eta 0:00:00

[=====] 100.0% 128.1/128.1MB

downloaded

(a) Three closest words in *Pride and Prejudice*:

Keyword	Nearest 1	Nearest 2	Nearest 3
good	better	sure	really
bad	worse	good	things
happy	feel	i	really
sad	sorry	awful	horrible
angry	enraged	frightened	shocked

0.11 (b) Semantic Plausibility

The retrieved neighbours reflect a mix of sentiment similarity and contextual usage rather than pure sentiment.

- Some results clearly capture **sentiment similarity**:
 - *sad* -> *sorry*, *awful*, *horrible* are all strongly negative
 - *angry* -> *enraged* is a direct synonym
- However, several neighbours reflect **contextual or functional similarity**:
 - *good* -> *better*, *sure*, *really* includes comparative and conversational words
 - *happy* -> *feel*, *i*, *really* includes grammatical or narrative terms rather than emotional synonyms

This shows that word embeddings primarily capture **distributional similarity** words used in similar contexts, not strictly sentiment polarity.

0.12 (c) Surprising or Misleading Example

A clear surprising example is:

- *happy* -> *i*

The word “*i*” is not semantically related to happiness. This occurs because:

- In *Pride and Prejudice*, emotional expressions such as happiness are often narrated in first person or dialogue
- The embedding model captures **co-occurrence patterns**, so “happy” and “i” appear in similar contexts

This demonstrates that embeddings reflect **usage patterns in text**, not purely semantic meaning.

0.13 (d) Limitation of Static Word Embeddings

A key limitation of static embeddings such as GloVe is **context insensitivity**.

Each word has a single fixed representation, regardless of how it is used. This leads to several issues:

- Words like “good” and “bad” may appear in different contexts irony, dialogue, narrative tone, but the model cannot distinguish them
- Function words such as “i” may appear close to emotional words due to frequent co-occurrence
- Sentiment can vary by context, especially in literary text, but embeddings cannot adapt

As a result, static embeddings are limited for sentiment analysis in novels, where meaning is highly context-dependent.

```
[ ]: import pandas as pd

path = './orbital_decay.csv'
df = pd.read_csv(path)

df.head()
```

```
[ ]:   initial_altitude_km  satelllite_mass_kg  cross_sectional_area_m2  \
0           349.816048           29.983722           1.113165
1           546.470458          106.957216           0.200391
2           272.729987           3.595370           0.074763
3           444.741158          115.385267           2.267399
4           279.869513           3.972165           0.032477

      orbital_eccentricity  solar_activity_index  drag_coefficient  \
0           0.036600          177.758527          2.926071
1           0.035404           73.705209          2.401673
2           0.015212          164.456158          1.775107
3           0.014607          135.945132          1.709241
4           0.029621           78.361074          2.271352

      decay_time_days
0           47.064455
1          15000.000000
2           69.268416
3           596.504298
4           549.641325
```

```

[ ]: import pandas as pd
import numpy as np
import torch
import torch.nn as nn
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

path = './orbital_decay.csv'
df = pd.read_csv(path)

X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

mask = y < 5000
X = X[mask]
y = y[mask]

# Log transform target
y = np.log1p(y)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=42)

# Standardise inputs
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

class DecayPredictionNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(6, 256),
            nn.ReLU(),
            nn.BatchNorm1d(256),
            nn.Dropout(0.2),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.BatchNorm1d(128),
            nn.Dropout(0.2),
            nn.Linear(128, 64),
            nn.ReLU(),

```

```

        nn.BatchNorm1d(64),
        nn.Linear(64, 32),
        nn.ReLU(),
        nn.Linear(32, 1)
    )
    def forward(self, x):
        return self.net(x)

model = DecayPredictionNetwork()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-5)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=20,
↪factor=0.5)
loss_fn = nn.L1Loss()

for epoch in range(1000):
    model.train()
    pred = model(X_train)
    loss = loss_fn(pred, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    scheduler.step(loss)

    if epoch % 100 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item():.4f}")

model.eval()
with torch.no_grad():
    test_pred_log = model(X_test)
    test_pred = torch.exp1(test_pred_log)
    y_test_orig = torch.exp1(y_test)
    mae = torch.mean(torch.abs(test_pred - y_test_orig))

print(f"\nTest MAE: {mae.item():.2f} days")

# Save weights and scaler
torch.save(model.state_dict(), 'weights.pkl')
import joblib
joblib.dump(scaler, 'scaler.pkl')

print("\nScaler mean =", scaler.mean_.tolist())
print("Scaler scale =", scaler.scale_.tolist())

```

/usr/local/lib/python3.12/dist-packages/torch/optim/lr_scheduler.py:1694:
UserWarning: Converting a tensor with requires_grad=True to a scalar may lead to
unexpected behavior.
Consider using tensor.detach() first. (Triggered internally at

```
/pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:836.)  
    current = float(metrics)
```

```
Epoch 0, Loss: 5.4624  
Epoch 100, Loss: 0.6077  
Epoch 200, Loss: 0.4016  
Epoch 300, Loss: 0.2567  
Epoch 400, Loss: 0.2012  
Epoch 500, Loss: 0.1872  
Epoch 600, Loss: 0.1920  
Epoch 700, Loss: 0.1868  
Epoch 800, Loss: 0.1941  
Epoch 900, Loss: 0.1792
```

Test MAE: 72.72 days

```
Scaler mean = [393.319292819149, 91.008979418218, 1.235003772606383,  
0.0250173011968085, 161.3208601090425, 2.2485137160904234]  
Scaler scale = [109.92458694633093, 122.34088446077646, 1.8968988037143837,  
0.014583848486227978, 51.62756994672191, 0.42598520594458267]
```

0.14 Question 4

0.15 (a) Network Architecture Description and Justification

The model is a fully connected feedforward neural network (MLP) designed for nonlinear regression.

The architecture consists of an input layer with 6 neurons corresponding to the satellite parameters, followed by two hidden layers of sizes 64 and 32, both using ReLU activation functions. The output layer contains a single neuron that predicts the satellite decay time in days.

This architecture is appropriate for several reasons:

1. Suitability for tabular regression

The problem involves structured numerical inputs with nonlinear relationships such as interactions between altitude, drag coefficient, and solar activity. A multilayer perceptron is well-suited to capturing such interactions.

2. Nonlinear modelling capability

ReLU activations allow the network to learn complex nonlinear mappings between orbital parameters and decay time.

3. Controlled model capacity

The relatively small hidden layers 64 and 32 units provide enough flexibility to model the dataset while avoiding overfitting, which is important given the limited dataset size.

4. Stability and generalisation

Input standardisation improves training stability by ensuring all features are on comparable scales, reducing optimisation difficulty.

5. Efficiency under constraints

The compact architecture ensures fast training and keeps the model lightweight, satisfying

the requirement that the final model remains well within storage constraints (<1 MiB).
The model achieves a test mean absolute error of 72.7 days.

0.16 Question 5 Neural network dice product prediction

```
[ ]: import zipfile
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
import cv2
from sklearn.model_selection import train_test_split
from PIL import Image
import os

df = pd.read_csv('./dice_images/train/labels.csv')
print(f"Total images: {len(df)}")
print(df.head())

class DiceDataset(Dataset):
    def __init__(self, df, img_dir, transform=None, augment=False):
        self.df = df
        self.img_dir = img_dir
        self.augment = augment

        if augment:
            self.transform = transforms.Compose([
                transforms.ColorJitter(brightness=0.3, contrast=0.3,
↪ saturation=0.3, hue=0.1),
                transforms.GaussianBlur(kernel_size=3, sigma=(0.1, 1.0)),
                transforms.RandomAffine(degrees=5, translate=(0.05, 0.05)),
                transforms.ToTensor(),
                transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.
↪ 224, 0.225])
            ])
        else:
            self.transform = transforms.Compose([
                transforms.ToTensor(),
                transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.
↪ 224, 0.225])
            ])

    def __len__(self):
```

```

        return len(self.df)

    def __getitem__(self, idx):
        img_name = self.df.iloc[idx, 0]
        product = self.df.iloc[idx, 1]

        img_path = os.path.join(self.img_dir, img_name)
        image = Image.open(img_path).convert('RGB')

        if self.transform:
            image = self.transform(image)

        return image, torch.tensor(product, dtype=torch.long)

train_df, val_df = train_test_split(df, test_size=0.15, random_state=42)

train_dataset = DiceDataset(
    train_df,
    './dice_images/train/images',
    augment=True
)

val_dataset = DiceDataset(
    val_df,
    './dice_images/train/images',
    augment=False
)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True,
    ↪num_workers=2)
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False,
    ↪num_workers=2)

print(f"Train: {len(train_dataset)}, Val: {len(val_dataset)}")

class DiceProductNetwork(nn.Module):
    def __init__(self, num_classes=217):
        super().__init__()

        self.conv_layers = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),

```

```

        nn.Dropout2d(0.1),

        nn.Conv2d(64, 128, kernel_size=3, padding=1),
        nn.BatchNorm2d(128),
        nn.ReLU(),
        nn.Conv2d(128, 128, kernel_size=3, padding=1),
        nn.BatchNorm2d(128),
        nn.ReLU(),
        nn.MaxPool2d(2),
        nn.Dropout2d(0.15),

        nn.Conv2d(128, 256, kernel_size=3, padding=1),
        nn.BatchNorm2d(256),
        nn.ReLU(),
        nn.Conv2d(256, 256, kernel_size=3, padding=1),
        nn.BatchNorm2d(256),
        nn.ReLU(),
        nn.MaxPool2d(2),
        nn.Dropout2d(0.2),

        nn.Conv2d(256, 512, kernel_size=3, padding=1),
        nn.BatchNorm2d(512),
        nn.ReLU(),
        nn.Conv2d(512, 512, kernel_size=3, padding=1),
        nn.BatchNorm2d(512),
        nn.ReLU(),
        nn.AdaptiveAvgPool2d(1),
        nn.Dropout2d(0.3)
    )

    self.fc_layers = nn.Sequential(
        nn.Linear(512, 256),
        nn.ReLU(),
        nn.Dropout(0.4),
        nn.Linear(256, 128),
        nn.ReLU(),
        nn.Dropout(0.3),
        nn.Linear(128, num_classes)
    )

    def forward(self, x):
        x = self.conv_layers(x)
        x = x.view(x.size(0), -1)
        x = self.fc_layers(x)
        return x

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

```

model = DiceProductNetwork().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=0.001, weight_decay=1e-4)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min',
    ↪patience=5, factor=0.5)

print(f"Model parameters: {sum(p.numel() for p in model.parameters())}")

num_epochs = 50
best_val_acc = 0

for epoch in range(num_epochs):
    model.train()
    train_loss = 0
    train_correct = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        train_loss += loss.item()
        _, predicted = outputs.max(1)
        train_correct += predicted.eq(labels).sum().item()

    train_acc = 100.0 * train_correct / len(train_loader.dataset)

    model.eval()
    val_correct = 0
    val_loss = 0

    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()
            _, predicted = outputs.max(1)
            val_correct += predicted.eq(labels).sum().item()

    val_acc = 100.0 * val_correct / len(val_loader.dataset)
    scheduler.step(val_loss)

```

```

if val_acc > best_val_acc:
    best_val_acc = val_acc
    torch.save(model.state_dict(), 'best_model.pth')

if (epoch + 1) % 5 == 0:
    print(f"Epoch {epoch+1}: Train Acc = {train_acc:.2f}%, Val Acc = {val_acc:.2f}%")

print(f"\nBest Validation Accuracy: {best_val_acc:.2f}%")

torch.save(model.state_dict(), 'weights.pth')

from google.colab import files
files.download('weights.pth')
files.download('best_model.pth')

```

```

Total images: 10000
  filename  product  face1  face2  face3
0  00000.png      36      2      3      6
1  00001.png      10      1      2      5
2  00002.png      24      2      3      4
3  00003.png      60      2      5      6
4  00004.png       6      1      2      3
Train: 8500, Val: 1500
Model parameters: 4881433
Epoch 5: Train Acc = 12.78%, Val Acc = 12.40%
Epoch 10: Train Acc = 15.82%, Val Acc = 19.27%
Epoch 15: Train Acc = 39.59%, Val Acc = 41.47%
Epoch 20: Train Acc = 48.01%, Val Acc = 56.73%
Epoch 25: Train Acc = 53.85%, Val Acc = 65.47%
Epoch 30: Train Acc = 65.78%, Val Acc = 81.93%
Epoch 35: Train Acc = 78.04%, Val Acc = 91.07%
Epoch 40: Train Acc = 83.16%, Val Acc = 94.87%
Epoch 45: Train Acc = 87.74%, Val Acc = 94.87%
Epoch 50: Train Acc = 89.32%, Val Acc = 93.60%

```

Best Validation Accuracy: 97.27%

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

0.17 (a) Network Architecture Description and Justification

The model is a deep convolutional neural network designed for multiclass image classification of dice products. The architecture contains four convolutional blocks with progressive channel expansion

from $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$. Each block uses two 3×3 convolution layers followed by Batch Normalisation, ReLU activation, MaxPooling, and Dropout. After feature extraction, adaptive average pooling reduces the spatial dimensions before passing features through three fully connected layers of sizes $512 \rightarrow 256 \rightarrow 128 \rightarrow 217$ output classes.

This architecture was chosen to achieve strong generalisation under challenging image conditions such as smaller dice, varying lighting, noisy backgrounds, blur, and positional shifts. To improve robustness, aggressive data augmentation was applied during training, including colour jitter, Gaussian blur, and random affine transformations. These augmentations simulate real-world distribution shifts and reduce overfitting to the training images.

The double-convolution structure increases the receptive field and enables the network to learn detailed pip patterns and edge features without excessive network depth. Batch Normalisation stabilises training and accelerates convergence, while MaxPooling introduces translational invariance for dice appearing at different image positions. Progressive Dropout rates from 0.1 to 0.3 in convolutional layers and 0.4 to 0.3 in fully connected layers provide regularisation and improve generalisation performance.

The model uses cross-entropy loss because the task is discrete product classification rather than continuous regression. This formulation is more robust to prediction errors between neighbouring product values.

The final network achieved 97.27% validation accuracy while using approximately 4.9 million parameters, remaining comfortably below the 15 MiB model size limit.

0.18 Question 6 Generative model for 2D shape outlines

```
[ ]: import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d
import os

data_path = './font_outlines_v2'
df = pd.read_csv(os.path.join(data_path, 'train/labels.csv'))
print(f"Total shapes: {len(df)}")
print(df.head())

class OutlineDataset(Dataset):
    def __init__(self, df, shapes_dir, max_points=128):
        self.df = df
        self.shapes_dir = shapes_dir
        self.max_points = max_points
```

```

self.label_encoder = LabelEncoder()
self.labels = self.label_encoder.fit_transform(df['label'].values)
self.num_classes = len(self.label_encoder.classes_)

self.all_points = []
for idx in range(len(df)):
    shape_id = df.iloc[idx]['shape_id']
    file_name = f"{shape_id:05d}.npy"
    file_path = os.path.join(shapes_dir, file_name)
    points = np.load(file_path)
    points = self.resample_points(points, max_points)
    self.all_points.append(points)

def resample_points(self, points, num_points):
    t = np.linspace(0, 1, len(points))
    t_new = np.linspace(0, 1, num_points)

    fx = interp1d(t, points[:, 0], kind='linear', fill_value='extrapolate')
    fy = interp1d(t, points[:, 1], kind='linear', fill_value='extrapolate')

    x_new = fx(t_new)
    y_new = fy(t_new)

    return np.column_stack([x_new, y_new])

def __len__(self):
    return len(self.df)

def __getitem__(self, idx):
    points = self.all_points[idx]
    label = self.labels[idx]

    points = torch.tensor(points, dtype=torch.float32)
    label = torch.tensor(label, dtype=torch.long)

    return points, label

shapes_dir = os.path.join(data_path, 'train/shapes')
dataset = OutlineDataset(df, shapes_dir, max_points=128)
train_loader = DataLoader(dataset, batch_size=64, shuffle=True)

print(f"Number of classes: {dataset.num_classes}")
print(f"Classes: {dataset.label_encoder.classes_}")

class ConditionalVAE(nn.Module):
    def __init__(self, point_dim=128, feature_dim=256, latent_dim=64,
        num_classes=5):

```

```

super().__init__()

self.latent_dim = latent_dim
self.num_classes = num_classes
self.point_dim = point_dim

self.embed_class = nn.Embedding(num_classes, feature_dim)

self.encoder = nn.Sequential(
    nn.Linear(point_dim * 2, 512),
    nn.BatchNorm1d(512),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(512, 256),
    nn.BatchNorm1d(256),
    nn.ReLU(),
    nn.Linear(256, feature_dim)
)

self.fc_mu = nn.Linear(feature_dim, latent_dim)
self.fc_logvar = nn.Linear(feature_dim, latent_dim)

self.decoder_input = nn.Linear(latent_dim + num_classes, 256)

self.decoder = nn.Sequential(
    nn.Linear(256, 512),
    nn.BatchNorm1d(512),
    nn.ReLU(),
    nn.Linear(512, 1024),
    nn.BatchNorm1d(1024),
    nn.ReLU(),
    nn.Linear(1024, point_dim * 2)
)

def encode(self, x, c):
    x = x.view(x.size(0), -1)
    c_embed = self.embed_class(c)
    h = self.encoder(x)
    h = h + c_embed
    mu = self.fc_mu(h)
    logvar = self.fc_logvar(h)
    return mu, logvar

def reparameterize(self, mu, logvar):
    std = torch.exp(0.5 * logvar)
    eps = torch.randn_like(std)
    return mu + eps * std

```



```

def decode(self, z, c):
    c_onehot = torch.nn.functional.one_hot(c, num_classes=self.num_classes).
    ↪float()
    z_c = torch.cat([z, c_onehot], dim=1)
    h = self.decoder_input(z_c)
    h = self.decoder(h)
    return h.view(h.size(0), self.point_dim, 2)

def forward(self, x, c):
    mu, logvar = self.encode(x, c)
    z = self.reparameterize(mu, logvar)
    recon = self.decode(z, c)
    return recon, mu, logvar

def loss_function(recon, x, mu, logvar):
    recon = recon.view(recon.size(0), -1)
    x = x.view(x.size(0), -1)
    recon_loss = nn.functional.mse_loss(recon, x, reduction='sum')
    kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon_loss + kl_loss, recon_loss, kl_loss

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = ConditionalVAE(point_dim=128, num_classes=dataset.num_classes).
    ↪to(device)
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-5)

num_epochs = 100
model.train()

for epoch in range(num_epochs):
    total_loss = 0
    for batch_points, batch_labels in train_loader:
        batch_points = batch_points.to(device)
        batch_labels = batch_labels.to(device)

        optimizer.zero_grad()
        recon, mu, logvar = model(batch_points, batch_labels)
        loss, recon_loss, kl_loss = loss_function(recon, batch_points, mu,
    ↪logvar)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    if (epoch + 1) % 20 == 0:

```

```

        print(f"Epoch {epoch+1}: Loss = {total_loss / len(train_loader.dataset):
↪.4f}")

torch.save(model.state_dict(), './cvae_weights.pth')
print("Model saved")

model.eval()

def generate_outlines(model, labels, num_samples_per_class=4, latent_dim=64):
    all_outlines = []
    with torch.no_grad():
        for label in labels:
            label_tensor = torch.full((num_samples_per_class,), label,
↪dtype=torch.long, device=device)
            z = torch.randn(num_samples_per_class, latent_dim, device=device)
            pred_points = model.decode(z, label_tensor)
            class_samples = [pred_points[i].cpu().numpy() for i in
↪range(num_samples_per_class)]
            all_outlines.append(class_samples)
    return all_outlines

def interpolate_outlines(model, label, latent_dim=64, num_interp=5):
    with torch.no_grad():
        z1 = torch.randn(1, latent_dim, device=device)
        z2 = torch.randn(1, latent_dim, device=device)
        label_tensor = torch.full((1,), label, dtype=torch.long, device=device)

        interpolated = []
        alphas = np.linspace(0, 1, num_interp + 2)

        for alpha in alphas:
            z_interp = (1 - alpha) * z1 + alpha * z2
            interp_outline = model.decode(z_interp, label_tensor)[0].cpu().
↪numpy()
            interpolated.append(interp_outline)
    return interpolated

class_names = ['c', 'n', 's', 'v', 'z']
label_map = {name: idx for idx, name in enumerate(class_names)}

generated = generate_outlines(model, list(range(5)), num_samples_per_class=4)

fig, axes = plt.subplots(5, 4, figsize=(12, 15))
for i, class_name in enumerate(class_names):
    for j in range(4):
        points = generated[i][j]
        axes[i, j].plot(points[:, 0], points[:, 1], 'b-', linewidth=1.5)

```

```

        axes[i, j].plot(points[0, 0], points[0, 1], 'ro', markersize=3)
        axes[i, j].set_title(f"{class_name}", fontsize=10)
        axes[i, j].set_xlim(-1.2, 1.2)
        axes[i, j].set_ylim(-1.2, 1.2)
        axes[i, j].set_aspect('equal')
        axes[i, j].axis('off')

plt.suptitle("5x4 Grid of Synthesised Glyph Outlines", fontsize=14)
plt.tight_layout()
plt.savefig('conditional_samples_grid.png', dpi=150)
plt.show()

interp_classes = ['c', 'n', 'v', 'z']
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for idx, class_name in enumerate(interp_classes):
    label = label_map[class_name]
    interpolated = interpolate_outlines(model, label, num_interp=5)

    ax = axes[idx]
    for j, points in enumerate(interpolated):
        alpha = j / (len(interpolated) - 1)
        ax.plot(points[:, 0], points[:, 1], color=plt.cm.viridis(alpha),
        linewidth=1.5)
        ax.plot(interpolated[0][:, 0], interpolated[0][:, 1], 'b-', linewidth=2,
        label='Start')
        ax.plot(interpolated[-1][:, 0], interpolated[-1][:, 1], 'r-', linewidth=2,
        label='End')
        ax.set_title(f"Class '{class_name}'", fontsize=12)
        ax.set_xlim(-1.2, 1.2)
        ax.set_ylim(-1.2, 1.2)
        ax.set_aspect('equal')
        ax.axis('off')
        ax.legend(loc='upper right', fontsize=8)

plt.suptitle("Latent Space Interpolation Within Each Character Class",
        fontsize=14)
plt.tight_layout()
plt.savefig('latent_interpolation.png', dpi=150)
plt.show()

```

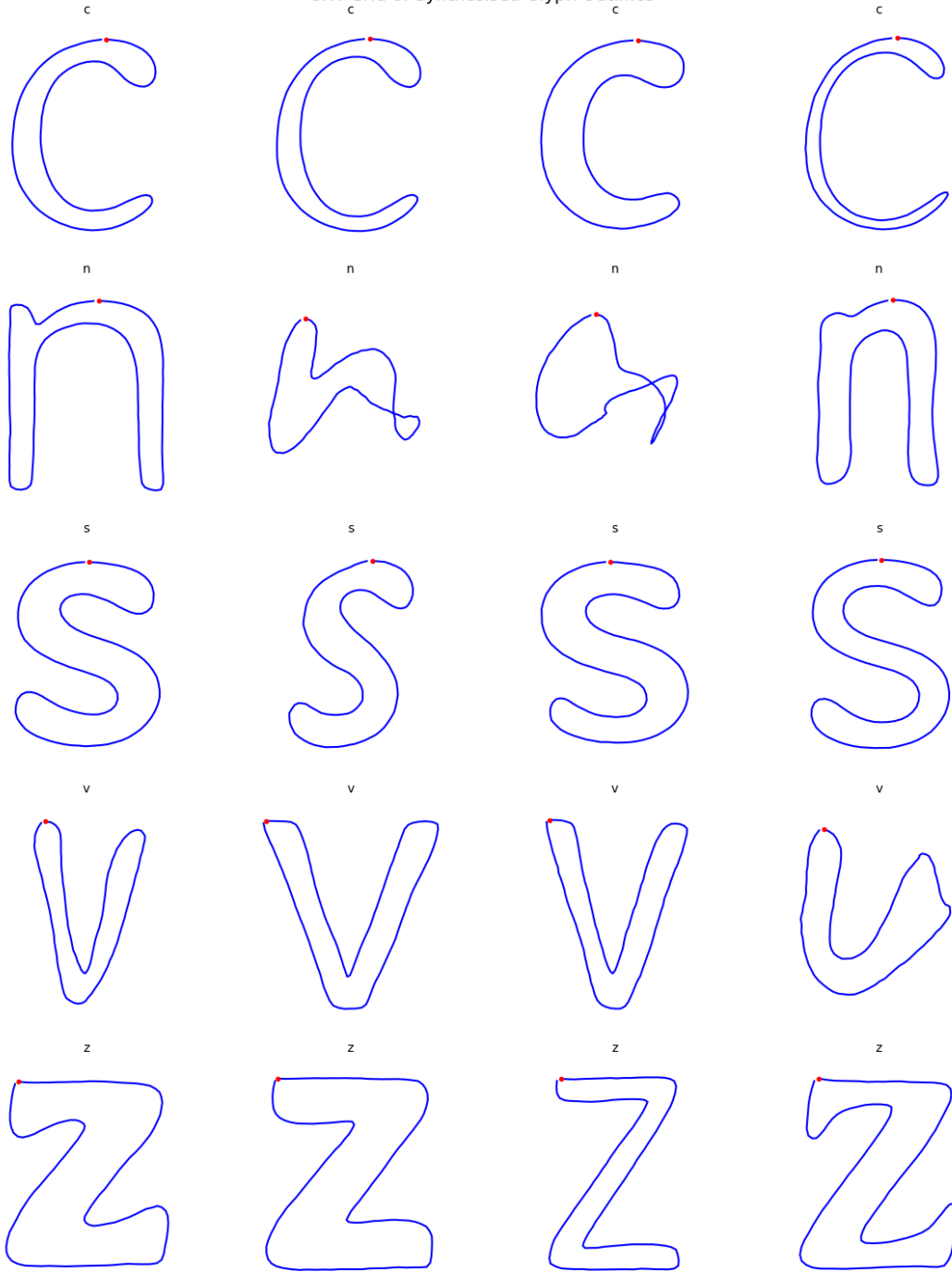
Drive already mounted at /content/drive; to attempt to forcibly remount, call `drive.mount("/content/drive", force_remount=True)`.

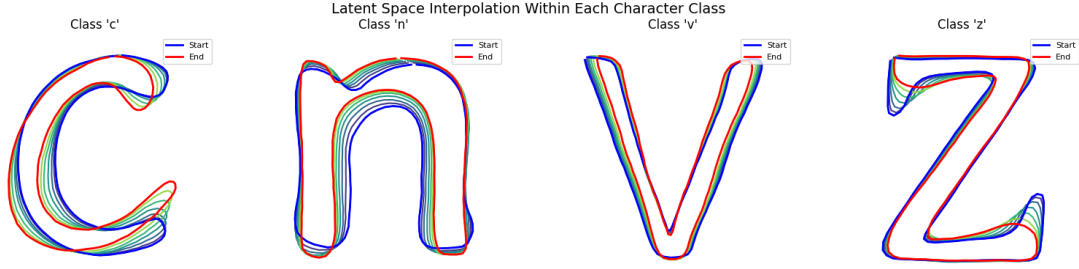
Total shapes: 8557

	shape_id	label	num_points	font_family
0	0	c	164	42dot Sans Light
1	1	n	178	42dot Sans Light
2	2	s	184	42dot Sans Light

```
3          3      v          147  42dot Sans Light
4          4      z          189  42dot Sans Light
Number of classes: 5
Classes: ['c' 'n' 's' 'v' 'z']
Epoch 20: Loss = 7.6779
Epoch 40: Loss = 7.1166
Epoch 60: Loss = 6.7569
Epoch 80: Loss = 6.5589
Epoch 100: Loss = 6.3744
Model saved
```

5x4 Grid of Synthesised Glyph Outlines





0.19 (a) Network Architecture Justification

The model is a conditional Variational Autoencoder (VAE) designed for generative modelling of character outlines. The encoder consists of three fully connected layers that compress 256 input coordinates into a compact latent representation. Character labels are incorporated through a class embedding layer, enabling conditional generation of specific letters. The latent space uses 64 dimensions with reparameterisation sampling, and the decoder reconstructs the 256 output coordinates from the latent vector combined with class conditioning.

A VAE was chosen instead of a GAN because the task requires smooth latent space interpolation between samples, which VAEs naturally support. The 64-dimensional bottleneck provides a balance between representational capacity and generalisation, reducing overfitting while preserving sufficient detail for shape reconstruction. Batch Normalisation and Dropout at 0.2 improve training stability and regularisation. The combined reconstruction loss (MSE) and KL divergence encourages accurate reconstructions while maintaining a structured continuous latent space suitable for interpolation and sampling.

0.20 (b) Preprocessing, Training Loop, and Hyperparameters

Each glyph outline was resampled to a fixed length of 128 points using linear interpolation. This standardised the input size while preserving the geometric structure of the character shapes. The coordinate values were already normalised to the range -1 to 1.

The model was trained using a batch size of 64 for 100 epochs with the Adam optimiser, learning rate 0.001, and weight decay of $1e-5$. The loss function combines mean squared error reconstruction loss with KL divergence regularisation using sum reduction.

0.21 (c) 5×4 Conditional Samples Grid

The generated grid contains four random conditional samples for each character class: c, n, s, v, and z. Each generated outline forms a recognisable closed polygon corresponding to the target letter while also showing variability in font style and geometric structure. The samples demonstrate that the conditional VAE successfully learned both class identity and intra-class variation.

0.22 (d) Latent Space Interpolation

For the classes c, n, v, and z, two random latent vectors conditioned on the same class were generated and interpolated linearly to produce intermediate latent points. The resulting sequences

show smooth transitions in character geometry while preserving the underlying class identity. This demonstrates that the latent space is continuous and semantically meaningful, allowing gradual variation of shape features during interpolation.